Production Order List — CSV Export — Function Design (機能設計)

> **Revision of 2026-10-24 (WI-017).** This file supersedes `002_DD-FN-CSV_製造指示一覧.md`, which stays unchanged as the earlier approved revision. Change: the production-order notes limit is 1000 characters (REQ-090, supersedes the 500 of REQ-016); see `work-items/WI-017/`.

002_DD-FN-CSV — used by 002_DD-CSV and 002_DD-API-CSV (API-PO-05); implements 002_BD-CSV FN-041 and FN-042; requirements REQ-085–REQ-088.

Document control (改版履歴)

Field	Value
Document ID	002_DD-FN-CSV
System name	ProductionManagementAI
Subsystem name	Production orders
Work item	WI-016
Created by	Claude (for ThongTM)
Created date	2026-10-07
Last updated by	Claude (for ThongTM)
Last updated date	2026-10-24

Version	Date	Author	Revision content
1	2026-10-07	Claude (for ThongTM)	Initial creation
2	2026-10-24	Claude (for ThongTM)	WI-017 (REQ-090–REQ-092): notes limit 500 → 1000 characters. Export row `Notes` length ≤ 1000. No processing change. Dated file name per WI-017 DEC-002

Overview and method index

Field	Value
Module name	`ProductionOrderService` (export), `IProductionOrderRepository` (export query), `ProductionOrderCsvWriter`, `CsvExportResult`, `IPlantClock` (one addition)
Overview	Server side of API-PO-05: validate, count within a read snapshot, refuse over 10,000, stream the rows as safe CSV, log and trace the export

No	Method name	Overview	Notes
1	`ProductionOrderService.ExportAsync`	Product check, snapshot, count, row limit, telemetry; returns an open export or an error result	Application; new
2	`IProductionOrderRepository.BeginReadSnapshotAsync` / `StreamExportRowsAsync`	Read-only REPEATABLE READ transaction; filtered, sorted, unpaged row stream	Port + Infrastructure; new. `CountOrdersAsync` reused
3	`ProductionOrderCsvWriter.WriteAsync` / `FormatRow` / `Field`	BOM, header row, row formatting, RFC 4180 quoting, formula neutralising	Application; new; pure apart from the output stream
4	`CsvExportResult.ExecuteResultAsync`	Sets the API-PO-05 headers, runs the writer, aborts the connection on a mid-stream failure, disposes the snapshot	Api; new
5	`IPlantClock.ToPlantTime`	UTC instant → plant-local `DateTime`	Port + `PlantClock`; additive

Shared utility references

No	Name	Overview	Notes
1	`ProductionOrderListQuery.TryCreate`	The list's validator; the export builds a `ProductionOrderListRequest` with `Page` and `PageSize` null	Unchanged
2	`ProductionOrderQueryExtensions.ApplyFilters` / `ApplySort`	The list's predicates and sort-key mapping (order number as tie-breaker)	Unchanged; shared so the export can never disagree with the screen
3	`ProductionOrderProblems.ToActionResult`	`Result` → Problem Details (400/422/…)	Unchanged
4	`ProductionOrderTelemetry`	Activity source, meter, outcome names	Two instruments added (method 1)
5	`ProductionOrderMessages` (`Msg`)	Message ID constants	Adds `ExportLimitExceeded = "MSG-E024"`

Request data

None — this module calls no other API or service; it reads the database through the repository port.

Response / value mapping

The CSV columns and their value mappings are defined once in 002_DD-API-CSV ("Response body (CSV)") and 002_BD-CSV M-11–M-15; method 3 implements them. The shared row type:

No	Name	Variable name	Type	Length	Required	Value mapping	Notes
1	Order number	`OrderNumber`	string	13	yes	—	
2	Product SKU / name / unit	`ProductSku`, `ProductName`, `ProductUnit`	string	50 / 200 / —	yes	—	
3	Line	`Line`	`OrderLineResponse?`	—	no	M-14	Same projection as the list row
4	Quantity	`Quantity`	decimal	—	yes	M-12	
5	Due date	`DueDate`	`DateOnly`	—	yes	M-13	
6	Status	`Status`	`ProductionOrderStatus`	—	yes	M-11	
7	Notes	`Notes`	string?	≤ 1000	no	—	Not in the list row; added for the export (DEC-003)
8	Created / updated / completed	`CreatedAtUtc`, `UpdatedAtUtc`, `CompletedAtUtc`	`DateTimeOffset` / `DateTimeOffset` / `DateTimeOffset?`	—	—	M-13	

`ProductionOrderExportRow` is a new record beside `ProductionOrderListRow`; the list row is not widened, so the list's projection (002_DB) is unchanged.

Method design

1. `ProductionOrderService.ExportAsync`

Field	Value
Description	Prepare an export for a validated query: check the product filter, open a read snapshot, count, apply the row limit and return the open export (count + row stream + plant time), or a `Result` error
Return type	`Task<Result<ProductionOrderExport>>`
Created by / date	Claude / 2026-10-07
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`ProductionOrderListQuery`	`query`	Validated by `TryCreate` (paging fields at their defaults, unused)
2	`string?`	`userId`	Caller's user ID from the controller (`ClaimTypes.NameIdentifier`), for the log entry only
3	`CancellationToken`	`cancellationToken`	Request aborted

Return value

Type	Name	Description
`Result<ProductionOrderExport>.Ok`	export	`ProductionOrderExport : IAsyncDisposable` with `int Count` , `DateTime PlantNow` , `DateOnly PlantToday` , `IAsyncEnumerable<ProductionOrderExportRow> Rows` , and the snapshot it owns. The caller (method 4) must dispose it
`Result<…>.Invalid`	—	`productId: MSG-E002` (as the list)
`Result<…>.RuleViolation`	—	`MSG-E024` , count > `ProductionOrderExport.MaxRows` (10,000, DEC-005)

Processing overview: mirrors `ListAsync` (same activity tags, same product check) but without paging and inside a read-only REPEATABLE READ transaction, so `Count` (sent as `X-Total-Count`) and the streamed rows come from one snapshot even while other users save orders. No row is read before the limit check passes. The snapshot is disposed here on every non-Ok path and by method 4 on the Ok path.

Processing flow

Step	Description	Calls
1	Start activity `ProductionOrder.Export`; set the list's query tags (`SetQueryTags`, page tags omitted)	`ProductionOrderTelemetry.Source`
2	Product filter set and not found → `Invalid { productId: [MSG-E002] }`; outcome `validation_failed`	`ProductExistsAsync` (existing)
3	Open the snapshot	method 2 `BeginReadSnapshotAsync`
4	`count = CountOrdersAsync(query)`	existing
5	`count > 10,000` → dispose snapshot; `RuleViolation(MSG-E024)`; outcome `rule_violation`; log 2005 with outcome `limit`	—
6	Read `PlantNow = ToPlantTime(utcNow)` and `PlantToday` once, so the file name, overdue column and every row agree	method 5
7	Return `Ok(new ProductionOrderExport(count, PlantNow, PlantToday, StreamExportRowsAsync(query), snapshot))`	method 2
8	On an unexpected exception before step 7: dispose snapshot, outcome `error`, `LogUnexpectedFailure(…, "export")`, rethrow (→ 500 `MSG-E013`)	—

Telemetry (additions to `ProductionOrderTelemetry`):

Instrument	Name	Tags	Recorded
Counter	`pmai.production_orders.exported`	`outcome` = `success` | `validation_failed` | `rule_violation` | `error` | `cancelled`	Once per export: steps 2/5/8 here, success/error/cancelled in method 4
Histogram	`pmai.production_orders.export_rows`	—	Rows written, on success (method 4)
Activity	`ProductionOrder.Export`	the list's filter/sort tags, `result.total`, `outcome`	Spans the service call and the streaming (method 4 ends it)
Log	EventId 2005 `ProductionOrdersExported`, Information: `Production orders export by {UserId}: {Outcome}, {RowCount} rows in {DurationMs} ms`	—	One entry per export; no order contents, notes or filter values are logged (002_BD-CSV observability)

2. `IProductionOrderRepository.BeginReadSnapshotAsync` and `StreamExportRowsAsync`

Field	Value
Description	`BeginReadSnapshotAsync(ct) → IProductionOrderTransaction`: `BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY` on the context's connection. `StreamExportRowsAsync(query) → IAsyncEnumerable<ProductionOrderExportRow>`: the list query without `Skip` / `Take`, projecting the export row
Return type	`Task<IProductionOrderTransaction>`; `IAsyncEnumerable<ProductionOrderExportRow>`
Created by / date	Claude / 2026-10-07

Field	Value
Last modified by / date	—

Arguments: `ProductionOrderListQuery query`, `CancellationToken cancellationToken` (via `WithCancellation`).

Return value: see Description. `CountOrdersAsync` (existing) runs inside the same transaction because both use the scoped `DbContext` connection.

Processing overview: `db.ProductionOrders.AsNoTracking().ApplyFilters(query).Join(products).ApplySort(query.Sort, query.Direction).Select(new ProductionOrderExportRow(…, line subquery as the list, Notes, CreatedAtUtc, UpdatedAtUtc, CompletedAtUtc)).AsAsyncEnumerable()`. EF Core streams with a data reader, so at most one row is materialised at a time (no `ToListAsync`). The query uses the filter and sort indexes of 002_DB and WI-004; only the `LIMIT/OFFSET` is gone, and the row count is bounded by method 1 step 5. `IsolationLevel.RepeatableRead` and `READ ONLY` are PostgreSQL-native (`pmai_app` needs no extra grant for either). The restricted app login keeps `SELECT` only on these tables; no permission change.

Processing flow

Step	Description	Calls
1	Begin `RepeatableRead` transaction, then `SET TRANSACTION READ ONLY`	`db.Database.BeginTransactionAsync(IsolationLevel.RepeatableRead)`
2	Build the query: `ApplyFilters` → join `products` → `ApplySort` → project	`ProductionOrderQueryExtensions` (unchanged)
3	Return `AsAsyncEnumerable()` — executed when method 3 enumerates it	—

3. `ProductionOrderCsvWriter`

Field	Value
Description	Static, culture-invariant writer for the API-PO-05 body. `WriteAsync(export, Stream output, ct)` writes BOM, header row and every row; `FormatRow(row, plantToday, toPlantTime)` and `Field(string?, bool isText)` are public for unit tests
Return type	`Task<int>` (rows written); `string`; `string`
Created by / date	Claude / 2026-10-07
Last modified by / date	—

Arguments

No	Type	Name	Description
1	`ProductionOrderExport`	`export`	From method 1
2	`Stream`	`output`	`HttpResponse.Body` (method 4); a `MemoryStream` in unit tests
3	`CancellationToken`	`cancellationToken`	`HttpContext.RequestAborted`

Return value

Type	Name	Description
`int`	rows	Rows written; equals `export.Count` because both come from one snapshot. A mismatch is logged as a warning (it would indicate a snapshot defect), not hidden

Processing overview and exact rules:

- **Encoding:** write `EF BB BF` explicitly, then a `StreamWriter` with `new UTF8Encoding(false)` (no second BOM), `NewLine = "\r\n"`, async writes only (Kestrel disallows synchronous I/O). Flush every 500 rows so the browser receives data while the query runs; `FlushAsync` at the end.
- **Header row:** the 13 Japanese headers of 002_DD-API-CSV, in order.
- **Values** (M-11–M-15): status labels from a fixed `switch` (`下書き`, `進行中`, `完了`, `取消`); quantity `ToString("0.###", CultureInfo.InvariantCulture)` (scale ≤ 3, no grouping, no trailing zeros); due date `yyyy/MM/dd`; timestamps `toPlantTime(utc).ToString("yyyy/MM/dd HH:mm", InvariantCulture)`, empty when null; overdue `納期遅れ` when `DueDate < plantToday && Status is Draft or InProgress` (the same predicate as `ProductionOrderListMapper.ToListItem`, extracted to one shared `IsOverdue` helper so the two cannot drift); line `{code} — {name}` plus `(使用停止)` when `!IsActive`, empty when null.
- **`Field(value, isText)`:** null → empty. If `isText` and the value starts with `=`, `+`, `-`, `@`, `\t` or `\r`, prefix `'` (OWASP CSV injection; CWE-1236). Then, if it contains `,`, `"`, `\r` or `\n`, wrap in `"` and double each `"` (RFC 4180). Text columns: 1, 2, 3, 4, 6, 8, 9, 10. Numeric/date columns (5, 7, 11–13) are produced by the formatters above and never prefixed. Line breaks inside notes are kept as stored (DEC-013).

Processing flow

Step	Description	Calls
1	Write BOM and header row	—
2	`await foreach` row in `export.Rows.WithCancellation(ct)`: write `FormatRow(…)` + CRLF; every 500 rows `FlushAsync`	method 2 stream
3	Final `FlushAsync`; return the row count	—

4. `CsvExportResult.ExecuteResultAsync`

Field	Value
Description	`IActionResult` returned by `ProductionOrdersController.Export` on `Ok`. Owns the `ProductionOrderExport` until the response ends
Return type	`Task`
Created by / date	Claude / 2026-10-07
Last modified by / date	—

Arguments: `ActionContext context` (framework). Constructed with the export, the logger and the start timestamp.

Processing overview: sets the 200 headers of 002_DD-API-CSV — `Content-Type`, `Content-Disposition` (built with `ContentDispositionHeaderValue`, `FileName` = `production-orders_{PlantNow:yyyyMMdd-HHmm}.csv`, `FileNameStar` = `製造指示一覧_{…}.csv`), `X-Total-Count = export.Count` — then calls method 3 on `Response.Body`. Errors before the first byte cannot happen here (all checks ran in method 1). If anything throws after headers were sent, the response cannot become a Problem Details any more, so it calls `HttpContext.Abort()`: the browser sees a network error and 002_DD-CSV shows MSG-E025, never a short file reported as success. In `finally` it disposes the export (ends the snapshot transaction), records the counter/histogram, ends the activity and writes log 2005.

Processing flow

Step	Description	Calls
1	Set status 200 and headers	—
2	`rows = await ProductionOrderCsvWriter.WriteAsync(export, Response.Body, RequestAborted)`	method 3
3	Success → counter `success`, histogram `rows`, log 2005 `success`	method 1 telemetry
4	`OperationCanceledException` with `RequestAborted` set → outcome `cancelled` (user left or view changed); nothing else to do	—
5	Any other exception → log `LogUnexpectedFailure(…, "export")`, outcome `error`, `HttpContext.Abort()`	—
6	`finally`: `await export.DisposeAsync()`	—

5. `IPlantClock.ToPlantTime`

Field	Value
Description	`DateTime ToPlantTime(DateTimeOffset utc)`: the plant-local wall-clock time of an instant (`TimeZoneInfo.ConvertTime(utc, zone).DateTime`), used for the timestamp columns and the file name (DEC-008)

Field	Value
Return type	`DateTime` (Kind `Unspecified`)
Created by / date	Claude / 2026-10-07
Last modified by / date	—

Additive to the port and `PlantClock`; `DateOf` keeps its behaviour and can delegate to it. Test doubles of `IPlantClock` in `tests/` gain the one member.

Unresolved decisions

None. Choices made here, within the approved design: REPEATABLE READ READ ONLY snapshot for count + rows; `ProductionOrderExportRow` separate from the list row; a shared `IsOverdue` helper; flush every 500 rows; log EventId 2005; metric names above. Recorded as WI-016 DEC-015 when this document is presented.